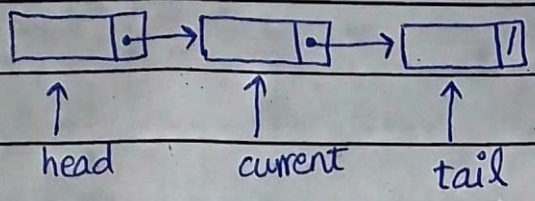
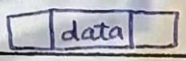
doubly linklist

single linklist:



doubly linklist:

```
#include<iostream>
using namespace std;
class Node
{  private:
    int data;
    Node *next;
    Node *prev;
    public:
    Node(): next(NULL), prev(NULL), size(0) {}
    void setData(int d) { data=d; }
    void setNext(Node *ptr) { next=ptr; }
    void setPrev(Node *ptr) { prev=ptr; }
    int getData() const { return data; }
    Node *getNext() const { return next; }
    Node *getPrev() const { return prev; }
};
```



```
class DLinkList
{   private:
    Node * head;
    Node * current;
    Node * tail;
    int size;
    public:
    DLinkList(): head(NULL), current(NULL),
                 tail(NULL), size(0) {}
//Add functions.
void addAtHead(int value)
{   if(head== NULL)
    {
        head= new Node();
        head-> setData(value);
        tail= current= head;
        size++;
    }
    else {
        Node * temp= new Node();
        temp-> setData(value);
        head-> setPrev(temp);
        temp-> setNext(head);
        head= temp;
        current= temp;
        size++;
        }
}
void addAtEnd(int value)
{
    if(head== NULL)
    {   // same as addOf//u01a }
```

```
else
{
    Node *temp = new Node();
    temp -> setData(value);
    temp -> setPrev (tail);
    tail -> setNext (temp);
    tail = temp;
    current = temp;
    size++;
}
}
void addAtCurrent (int value)
{   if(head == NULL)
    {
        // same as addAtHead
    }
    else
    {   Node *temp = new Node();
        temp -> setData(value);
        temp -> setNext (current -> getNext());
        current -> setNext (temp);
        temp -> setPrev (current);
        if (current == tail)
            tail = current = temp;
        else {
        (temp -> getNext()) -> setPrev (temp);
        current = temp; }
        size++;
    }
}
```

```
void removeFromHead()
{   if (head == NULL)
        return;
    Node * temp = head;
    head = head -> getNext();
    if (current == temp)
        current = head;
    delete temp;
    size--;
}
void removeFromEnd()
{   if (head == NULL)
        return;
    Node * temp = tail;
    tail = tail -> getPrev();
    if (current == temp)
        current = tail;
    delete temp;
    size--;
}
void removeFromCurrent()
{   if (head == NULL)
        return;
    if (current == head)
    { // same as removeFromHead() }
    else if (current == tail)
    { // same as removeFromEnd() }
    else { Node * Ptr1 = current -> getPrev();
           Node * Ptr2 = current -> getNext();
           ptr -> setNext(Ptr2); Ptr2 -> setPrev(Ptr1);
```

```
void next()
{   if(head!=NULL && current!=tail)
        current = current->getNext();
}
void back()
{   if(head!=NULL && current!=head)
        current = current->getPrev();
}
bool search(int value)
{   if(head==NULL)
        return false;
    Node *temp = head;
    While(temp!=NULL)
    {   if(temp->getData()==value) return true;
        cout << temp->getData() << " ";
        temp = temp->getNext();
    }
    return false;
}
void display()
{   if(head==NULL)
        return;
    Node *temp = head;
    while(temp!=NULL)
    {
        cout << temp->getData() << " ";
        temp = temp->getNext();
    }
}
};
```